VELLORE INSTITUTE OF TECHNOLOGY (VIT)

School of Computer Science Engineering and Information Systems (SCORE)

Department of Information Technology

FALL SEMESTER 2026–2027

CLOUDFORGE

Cloud Operations Platform for AWS

Architecture, Implementation, and Verification Report

COURSE DETAILS:

Course Code: BCSE355L | **Course Name:** Cloud Architecture Design

SUBMITTED BY:

TEAM: PiedPipers

Reg. No.	Student Name	Contact Email
24BIT0595	Mandar Joshi	mandar.pravinjoshi2024@vitstudent.ac.in
24BIT0057	Animesh Abhi Dubey	animesh.abhidubey2024@vitstudent.ac.in
24BIT0268	Vivek Choudhari	choudhari.vivek2024@vitstudent.ac.in

Date of Milestone Review: September 2026

Contents

1 Introduction and Problem Context **2**
1.1 Background and Problem Statement . . . 2
1.2 Project Objectives . . . 2
1.3 AWS Well-Architected Framework Alignment . . . 2

2 System Architecture and Technical Design **4**
2.1 System Architecture Overview . . . 4
2.2 System Architecture Diagram . . . 4
2.3 Event-Driven Ingestion Pipeline (AWS CDK) . . . 4

3 Completed Modules and Implementation Details **6**
3.1 Module 1: AWS Discovery and Multi-Region Live Synchronization . . . 6
3.2 Module 2: EC2 Operational Lifecycle Control . . . 6
3.3 Module 3: Cloud Health Scoring Engine . . . 6
3.4 Module 4: Dependency Graph and Blast Radius Calculation . . . 7
3.5 Module 5: Security Governance and CIS Auto-Remediation . . . 7
3.6 Module 6: Cost Analytics and Resource Optimization . . . 8
3.7 Module 7: Automated Operational Workflows . . . 8
3.8 Module 8: Native Rust Telemetry Daemon (`cloudforge-agent`) . . . 8

4 Verification and Functional Testing Matrix **9**
4.1 Role-Based Access Control Implementation . . . 9

5 Process and Control Flow Diagrams **11**
5.1 Cognito RBAC and JWT Authentication Lifecycle . . . 11
5.2 Resource Optimization State Machine . . . 11

6 Next Phase Engineering Roadmap **12**
6.1 Milestone 1: AWS Cost Explorer API Integration . . . 12
6.2 Milestone 2: Automated Incident Log Analysis and Script Generation . . . 12
6.3 Milestone 3: Multi-Account AWS Organizations Federation . . . 12
6.4 Milestone 4: Container Cluster Topology Visualization . . . 12
6.5 Milestone 5: Automated CI/CD Deployment Pipeline . . . 12

7 Conclusion **13**

Abstract

Managing Amazon Web Services (AWS) infrastructure across multiple regions involves operational complexity when using disparate native web consoles. **CloudForge** is a centralized Cloud Operations platform developed to consolidate real-time infrastructure discovery, dependency visualization, multi-region lifecycle management, security compliance auditing, cost optimization, and automated operational tasks into an integrated system.

The system is constructed as a monolithic full-stack application combining a React 18 TypeScript user interface with a Node.js 20 Express backend utilizing the AWS SDK v3, paired with a native infrastructure agent implemented in Rust using Axum and Bollard. This report documents the completed system modules, verified operational functionalities (including multi-region discovery across Amazon EC2, VPC, Subnets, Security Groups, RDS, S3, Lambda, and DynamoDB; dependency mapping with downstream impact calculation; and AWS CDK event pipelines), presents architectural diagrams, and outlines the engineering plan for subsequent phases.

1 Introduction and Problem Context

1.1 Background and Problem Statement

Cloud deployments frequently consist of hundreds of interconnected resources distributed across multiple AWS regions and accounts. Managing these environments using the standard AWS Management Console presents several operational challenges:

1. **Console Fragmentation:** Operators must navigate separate consoles for EC2, VPC, RDS, S3, CloudWatch, CloudTrail, and Cost Explorer, increasing the time required to correlate infrastructure events.
2. **Resource Dependency Complexity:** Infrastructure components are interconnected. Changes or outages in virtual networking or firewall configurations impact compute and database instances. Native tools do not provide unified relationship graphs that model failure propagation.
3. **Unused Resource Costs:** Stopped virtual instances with attached EBS storage, unattached storage volumes, and older storage tiers accumulate recurring charges without periodic review.
4. **Configuration and Compliance Drift:** S3 buckets with permissive access policies, unencrypted volumes, and unrestricted security group rules require continuous monitoring against baseline security benchmarks.

1.2 Project Objectives

CloudForge addresses these requirements through the following technical objectives:

- **Multi-Region Infrastructure Visibility:** Discover, index, and monitor AWS resources across multiple regions via AWS SDK v3 APIs.
- **Infrastructure Dependency Graph:** Render topological relationships among VPCs, Subnets, EC2 instances, RDS databases, and S3 buckets with automated failure impact calculation.
- **Composite Health Scoring:** Calculate an automated health index $(0 - 100)$ based on resource availability, security compliance checks, and resource utilization metrics.
- **Operational Lifecycle Management:** Enable authorized operators to perform instance actions (Start, Stop, Reboot, Snapshot) directly with state validation.
- **Cost Optimization:** Aggregate cloud expenditure estimates, identify underutilized resources, and apply storage optimization policies.
- **Security Compliance Evaluation:** Audit cloud resources against CIS AWS Foundations Benchmark criteria and provide automated remediation routines.
- **Event Auditing Pipeline:** Capture and record infrastructure changes from AWS CloudTrail, AWS Config, and Amazon EventBridge into an audit log.

1.3 AWS Well-Architected Framework Alignment

The platform architecture is designed according to the five pillars of the **AWS Well-Architected Framework**:

Pillar	Implementation in CloudForge
Operational Excellence	Automated operational workflows, system diagnostic routines, and structured audit logs.
Security	Amazon Cognito Role-Based Access Control (`Admin`, `CloudOps Engineer`, `Viewer`), AWS STS temporary credential verification, and CIS rule scanning.
Reliability	Multi-region data aggregation, dead-letter queues (SQS DLQ) on event ingestion, and multi-AZ failover testing.
Performance Efficiency	Asynchronous Node.js backend combined with a native Rust agent (`cloudforge-agent`) for low-latency host and container telemetry.
Cost Optimization	FinOps analytics engine identifying stopped EC2 instances and recommending S3 lifecycle archival transitions.

Table 1: CloudForge Alignment with AWS Well-Architected Pillars

2 System Architecture and Technical Design

2.1 System Architecture Overview

CloudForge uses a monolithic server architecture complemented by a native host telemetry agent:

- **Frontend Application (`src/client/`):** Implemented in React 18, TypeScript, Tailwind CSS, Lucide icons, and React Flow. Provides navigation, topology visualization, and operational controls.
- **Backend Monolith Server (`src/server/`):** Node.js Express application in TypeScript. Executes AWS SDK v3 calls, enforces authentication and access control, computes health scores, and serves static frontend bundles.
- **Native Telemetry Agent (`services/cloudforge-agent/`):** Standalone background daemon written in Rust using Tokio, Axum, Bollard, and Sysinfo. Communicates with `/var/run/docker.sock` to collect container metrics and process information.
- **Infrastructure as Code (`cdk/`):** AWS CDK stacks defining Amazon Cognito User Pools, DynamoDB tables, S3 audit buckets, EventBridge rules, SQS queues, and Lambda functions.

2.2 System Architecture Diagram

Figure 1 illustrates the multi-tier architecture of CloudForge.

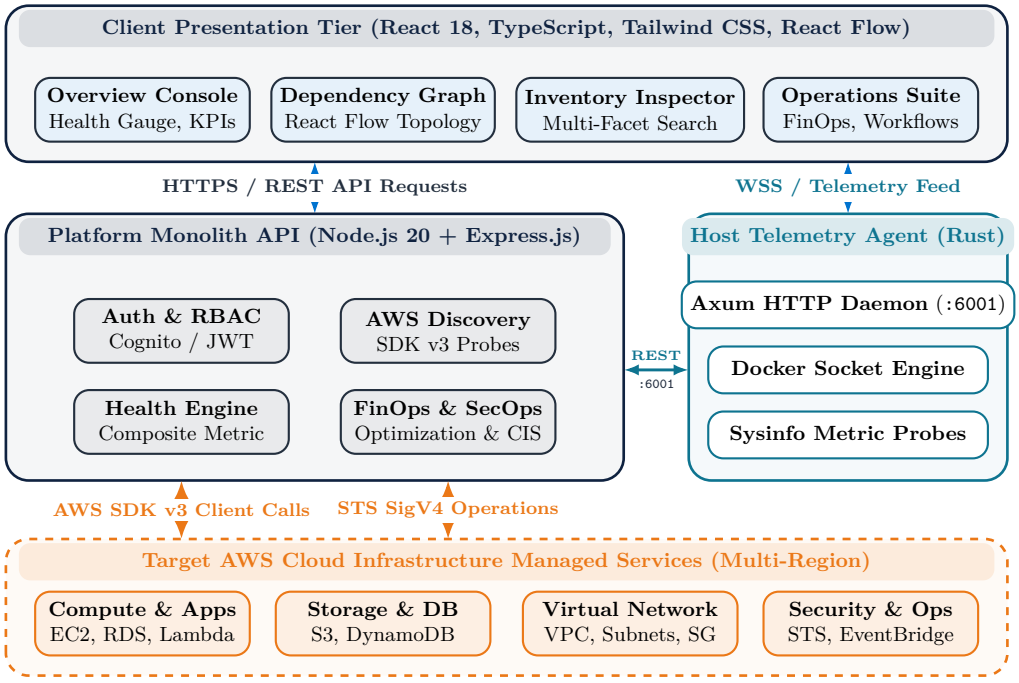


Figure 1: CloudForge End-to-End System Architecture

2.3 Event-Driven Ingestion Pipeline (AWS CDK)

To track configuration changes asynchronously, CloudForge implements an event pipeline using Amazon EventBridge, Amazon SQS FIFO, and AWS Lambda, defined via AWS CDK in `cdk/lib/event-pipeline-sta`

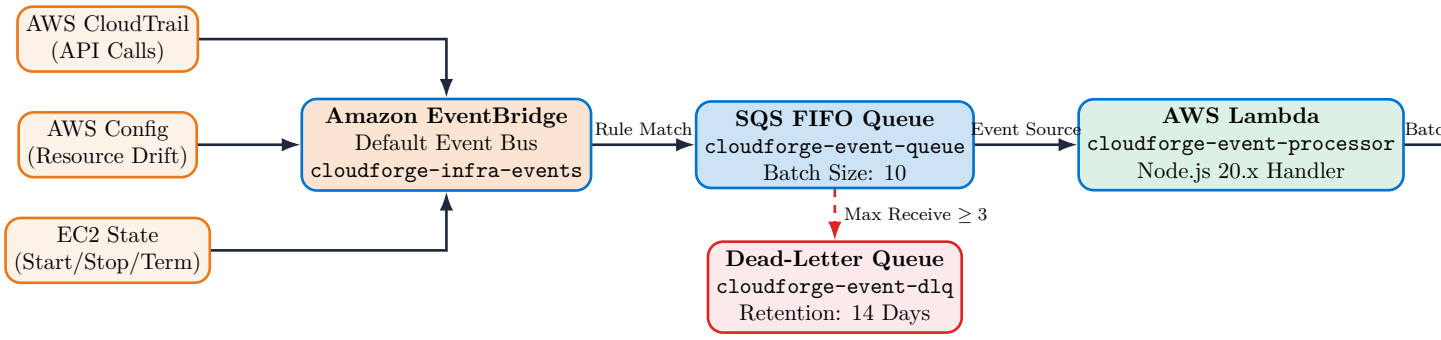


Figure 2: Asynchronous Event-Driven Infrastructure Auditing Pipeline

3 Completed Modules and Implementation Details

3.1 Module 1: AWS Discovery and Multi-Region Live Synchronization

Located in `src/server/services/awsDiscoveryService.ts`, this component handles connectivity and data retrieval using AWS SDK v3:

- **STS Identity Verification:** Uses `GetCallerIdentityCommand` to validate IAM credentials, retrieving Account ID, User ARN, and Session Token.
- **Multi-Region Scanning:** Queries regional endpoints in parallel (`us-east-1`, `ap-southeast-2`, `ap-south-1`, `us-west-2`, `eu-west-1`, `ap-northeast-1`).
- **Supported Resource Types:**
  - **Amazon EC2:** Instances, AMI types, state flags, VPC bindings, IP allocations, and tag sets.
  - **Amazon VPC and Subnets:** VPC IDs, CIDR blocks, Availability Zones, and route associations.
  - **Security Groups:** Firewall ingress/egress rules and port specifications.
  - **Amazon RDS:** DB instances, database engines, allocated storage, and encryption flags.
  - **Amazon S3:** Storage buckets, creation dates, BlockPublicAccess policies, and SSE-KMS settings.
  - **AWS Lambda:** Functions, runtimes, memory configurations, and execution handlers.
  - **Amazon DynamoDB:** Provisioned and on-demand table metadata.
- **Session Persistence:** Caches verified session metadata in `.aws_session_cache.json` for session restoration across server restarts.

3.2 Module 2: EC2 Operational Lifecycle Control

Enables administrators to execute management commands directly on compute instances:

- **Lifecycle Operations:** Supports `StartInstancesCommand`, `StopInstancesCommand`, and `RebootInstance`
- **State Validation:** Verifies the current instance state before sending commands to prevent conflicting transitions.
- **Event Logging:** Records executed operations with user identity, timestamp, and instance ID into `timelineService`.

3.3 Module 3: Cloud Health Scoring Engine

Located in `src/server/services/healthEngine.ts`, this engine evaluates the health of the discovered infrastructure and outputs a score $(0 - 100)$ and letter grade ($A+$ to F).

Health Score Mathematical Formulation

The composite Health Score (S_{health}) is calculated as:

$$S_{\text{health}} = \max\left(0,\ \min\left(100,\ 100 - \left[15 \cdot N_{\text{critical}} + 7 \cdot N_{\text{degraded}} + 8 \cdot N_{\text{sec_violations}}\right]\right)\right) \quad (1)$$

where:

- N_{critical} is the number of resources in failed or unreachable states.
- N_{degraded} is the count of stopped or throttled nodes.
- $N_{\text{sec_violations}}$ is the count of failed CIS benchmark checks.

When AWS credentials are not configured, the engine operates in **Host Telemetry Mode**, evaluating host CPU utilization and memory consumption from `telemetryEngine.ts`.

3.4 Module 4: Dependency Graph and Blast Radius Calculation

Located in `src/client/components/DependencyGraphView.tsx`, this module visualizes infrastructure topologies:

- **Node Visualization:** Custom nodes for VPCs, Subnets, Security Groups, EC2, RDS, S3, and Lambda, organized by service type and health status.
- **Edge Mapping:** Generates containment relationships (VPC → Subnet → EC2) and consumer relationships (EC2 → RDS, EC2 → S3 Bucket).
- **Failure Impact Simulation:** Identifies and highlights downstream dependent resources when a specific node or network boundary is selected.

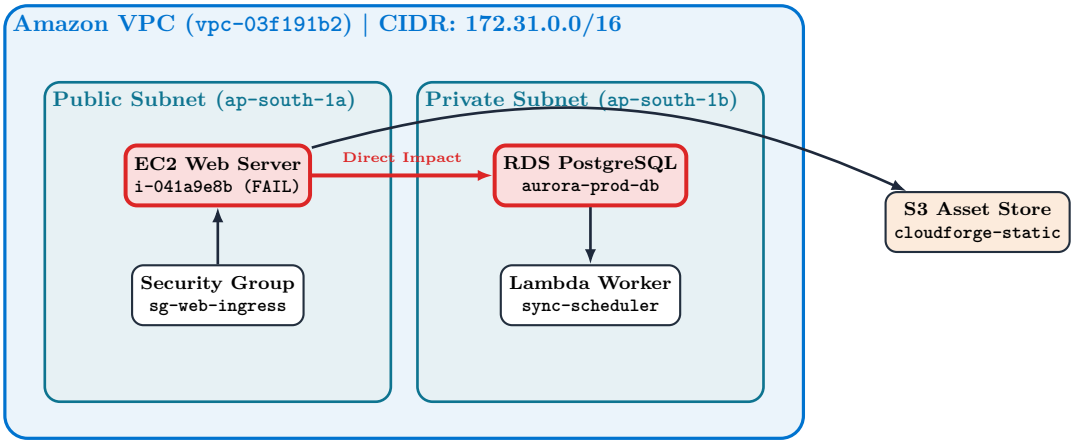


Figure 3: Infrastructure Dependency Graph and Failure Propagation Model

3.5 Module 5: Security Governance and CIS Auto-Remediation

Located in `src/server/services/securityService.ts`, this service scans resources for compliance:

- **S3 Public Access Checks:** Scans buckets for open access policies and missing public block settings.
- **KMS Encryption Audit:** Validates server-side encryption (SSE-KMS) configuration on data stores.

- **VPC Boundary Verification:** Verifies that compute instances are deployed within VPC private subnets.
- **Policy Remediation:** Applies compliance policies upon operator confirmation and records an audit log entry.

3.6 Module 6: Cost Analytics and Resource Optimization

Located in `src/server/services/costService.ts`, this module analyzes cloud expenditure:

- **Spend Projection:** Calculates estimated monthly expenditure and aggregates costs by service type.
- **Idle Resource Detection:** Identifies stopped EC2 instances with attached EBS storage volumes.
- **Storage Tier Recommendations:** Identifies infrequently accessed S3 buckets for lifecycle transitions to S3 Glacier.
- **Automated Optimization:** Executes snapshot and archival policies on confirmed idle resources.

3.7 Module 7: Automated Operational Workflows

Located in `src/server/services/workflowService.ts`, this module provides structured runbooks:

1. **Host System Diagnostics:** Evaluates host CPU load, available RAM, process counts, and runtime metrics.
2. **AWS Cloud Discovery Sync:** Refreshes multi-region inventory and validates STS caller identity.
3. **S3 Security Hardening:** Applies `PutPublicAccessBlock` on staging buckets.
4. **FinOps Resource Optimization:** Snapshots and archives storage volumes for stopped compute instances.
5. **Resilience and Latency Testing:** Injects synthetic latency across secondary availability zones to verify database failover behavior.
6. **Platform Health Benchmark:** Evaluates API response times across backend endpoints.

3.8 Module 8: Native Rust Telemetry Daemon (`cloudforge-agent`)

Located in `services/cloudforge-agent/`, this daemon provides local system and container telemetry:

- **Axum REST Service:** Listens on port 6001 with asynchronous request handling via Tokio.
- **Docker Socket Client:** Connects to `/var/run/docker.sock` via Bollard to retrieve container resource usage and execute container lifecycle operations (`start`, `stop`, `restart`).
- **Hardware Metric Probes:** Collects CPU usage, memory allocation, load averages (1m, 5m, 15m), and process tables via Sysinfo.

4 Verification and Functional Testing Matrix

The verification status of CloudForge endpoints and user interface views is detailed in Table 2.

Subsystem	Endpoint / View	Status	Verified Capability
Core API	`GET /api/ping`	**VERIFIED**	Healthcheck endpoint returning service status and version.
	`GET /api/aws/status`	**VERIFIED**	Returns STS Account ID, active compute regions, and host metrics.
	`POST /api/aws/connect`	**VERIFIED**	Validates credentials against AWS STS and caches active session.
Inventory	`GET /api/resources`	**VERIFIED**	Returns multi-region AWS resources with search and service filtering.
	`GET /api/resources/:id`	**VERIFIED**	Retrieves configuration, tags, and metrics for target resource.
	`POST /api/resources/:id/action`	**VERIFIED**	Executes EC2 `start`, `stop`, `reboot`, and EBS snapshot.
Health Engine	`GET /api/health`	**VERIFIED**	Computes dynamic score $(0-100)$, grade, and per-service breakdown.
	`HealthGauge.tsx`	**VERIFIED**	SVG circular gauge rendering overall score and grade badge.
Graph & Topology	`DependencyGraphView`	**VERIFIED**	Interactive React Flow graph with pan, zoom, and minimap.
	Failure Impact Mode	**VERIFIED**	Identifies and highlights downstream dependent nodes upon failure.
FinOps Engine	`GET /api/cost`	**VERIFIED**	Computes spend by service, daily trends, and idle compute savings.
	`POST /api/cost/optimize/:id`	**VERIFIED**	Applies storage optimization policies and logs audit event.
Security Posture	`GET /api/security`	**VERIFIED**	Audits S3 public access, KMS encryption, and VPC isolation.
	`POST /api/security/remediate/:id`	**VERIFIED**	Applies automated policy remediation and records audit event.
Automation	`GET /api/workflows`	**VERIFIED**	Lists available diagnostic, security, and FinOps workflows.
	`POST /api/workflows/:id/trigger`	**VERIFIED**	Executes multi-step runbooks with timing measurements.
Rust Agent	`GET /api/v1/telemetry`	**VERIFIED**	Streams host CPU, RAM, and load averages from port 6001.
	`GET /api/v1/resources`	**VERIFIED**	Container discovery via Docker Unix domain socket.

Table 2: CloudForge Subsystem Verification Matrix

4.1 Role-Based Access Control Implementation

CloudForge implements Role-Based Access Control (RBAC) via Amazon Cognito and JWT tokens:

- **Admin:** Unrestricted operational access, including EC2 lifecycle control, security remediation, and workflow execution.
- **CloudOps Engineer:** Operations and workflow execution; restricted from modifying critical IAM permissions.

- **Viewer:** Read-only access to dashboard metrics and resource inventory; mutation actions are disabled in the UI and rejected by server middleware (`src/server/middleware/rbac.ts`).

5 Process and Control Flow Diagrams

5.1 Cognito RBAC and JWT Authentication Lifecycle

Figure 4 illustrates the user authentication and authorization workflow.

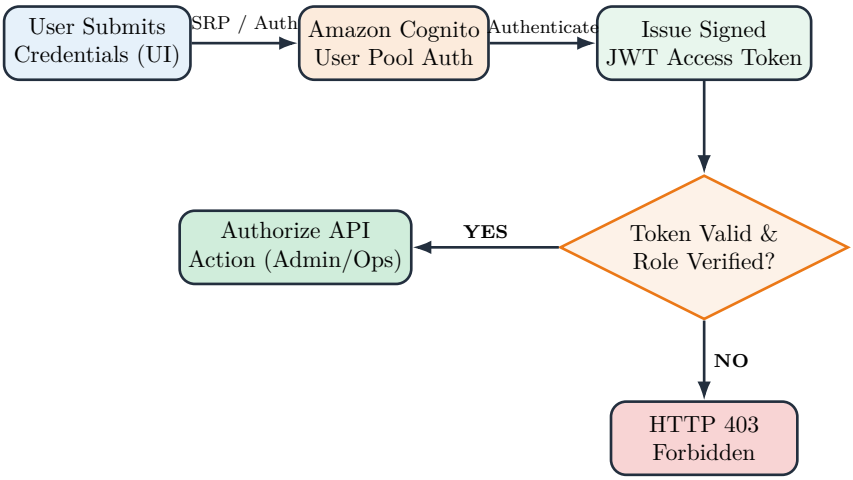


Figure 4: Cognito RBAC Authentication and Authorization Flowchart

5.2 Resource Optimization State Machine

Figure 5 shows the lifecycle for detecting and optimizing underutilized resources.

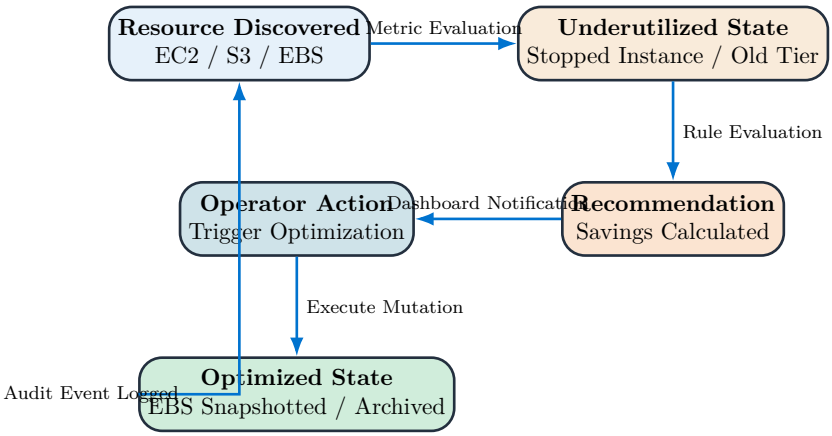


Figure 5: Resource Optimization Lifecycle State Machine

6 Next Phase Engineering Roadmap

Subsequent development will focus on expanding platform integration capabilities:

6.1 Milestone 1: AWS Cost Explorer API Integration

- Connect directly to `@aws-sdk/client-cost-explorer` to retrieve historical billing records via `GetCostAndUsageCommand`.
- Implement time-series statistical modeling to detect expenditure anomalies before the end of the billing period.

6.2 Milestone 2: Automated Incident Log Analysis and Script Generation

- **CloudTrail and CloudWatch Log Parsing:** Ingest structured and unstructured log entries during operational alerts.
- **Remediation Script Generation:** Generate executable AWS CLI and Python (Boto3) scripts corresponding to identified configuration errors.

6.3 Milestone 3: Multi-Account AWS Organizations Federation

- **Cross-Account AssumeRole:** Enable management of multiple spoke AWS accounts from a single management account using `sts:AssumeRole`.
- Provide account-level switching from the navigation interface.

6.4 Milestone 4: Container Cluster Topology Visualization

- Ingest Amazon EKS (Kubernetes) and Amazon ECS cluster metadata to visualize Pods, Services, and Ingress routing paths.
- Associate container resource usage metrics collected by the Rust telemetry agent with cluster topology nodes.

6.5 Milestone 5: Automated CI/CD Deployment Pipeline

- Configure GitHub Actions CI/CD workflows to run unit test suites, synthesize CDK templates, and deploy stacks to target AWS accounts.
- Implement automated end-to-end integration test runs covering all REST endpoints.

7 Conclusion

The CloudForge platform provides an integrated solution for monitoring, visualizing, and managing AWS cloud infrastructure. By unifying multi-region discovery, topological relationship mapping, failure impact analysis, security compliance checks, cost analytics, and automated workflows into a single system, CloudForge simplifies cloud management workflows. The addition of a native Rust agent demonstrates low-latency host and container telemetry collection without impacting application runtime performance.

References

[1] Amazon Web Services, "AWS Well-Architected Framework: Reliability, Security, and Operational Excellence Pillars," AWS Whitepapers, 2024.

[2] Center for Internet Security, "CIS Amazon Web Services Foundations Benchmark v3.0.0," CIS Security Benchmarks, 2023.

[3] M. Moritz and C. Ganter, "React Flow: A Highly Customizable Library for Building Node-Based UIs," Web Engineering Practice, 2024.

[4] Tokio Project Developers, "Asynchronous Runtime for the Rust Programming Language," https://tokio.rs, 2024.

[5] FinOps Foundation, "FinOps Framework: An Operating Model for Cloud Financial Management," Technical Principles, 2024.

[6] Amazon Web Services, "AWS Cloud Development Kit (AWS CDK) v2 Developer Guide," AWS Documentation, 2024.